A partial reproduction of Zep's temporal agent memory

Project: FTEC5660 Reproducibility Work, paper 12 **Paper:** Rasmussen et al., *Zep: A Temporal Knowledge Graph Architecture for Agent Memory* (2025) **Code:** getzep/graphiti, snapshot `fb60e59d4a208c99e7d0186ee6335d2a257f17f2`, version 0.31.2

1. Target and summary

I tested one concrete claim: dated conversation episodes can be turned into facts whose validity intervals preserve both a user's old and new state. I also measured whether those facts help answer questions compared with supplying the full conversation. The isolated modification removed `valid_at` and `invalid_at` from the *same* retrieved facts, leaving every other part of the answer pipeline unchanged.

The local synthetic behavior reproduces the temporal mechanism: Graphiti retains the old fact with an invalidation date and a new valid fact. However, on a 30-question public LongMemEval Oracle subset, this simplified implementation does not reproduce the paper's accuracy advantage over full context. This is informative negative evidence under materially different models, data, and ingestion granularity.

2. System and setup

The authors' open-source Graphiti engine ingests a dated episode, extracts entities and relation facts, resolves them against the existing graph, and records fact validity intervals. Search then returns relevant facts for a downstream answer model. I installed the authors' Python source snapshot without changing its source. I used DeepSeek's OpenAI-compatible chat API for extraction, resolution, answering, and public-data judging. The environment is Windows with Python 3.12, Kuzu 0.11.3 embedded graph storage, and a locally cached BAAI/bge-small-en-v1.5 embedding model (384 dimensions). DeepSeek decoding used model `deepseek-chat`, base URL `https://api.deepseek.com/v1`, temperature 0, up to 4096 output tokens for Graphiti operations and 128 for short answers.

The original paper used the Zep service, Neo4j, GPT-4o-mini for graph building, GPT-4o-mini/GPT-4o for answering, and BGE-m3 for embeddings and reranking. My edge-only search uses Graphiti's reciprocal-rank-fusion default with top 10 facts; no community summaries are built. Consequently the absolute scores and latency numbers below are **not directly comparable** to the paper's results.

3. Data and method

The first dataset contains 20 deterministic two-episode synthetic conversations across workplace, residence, project, device, and membership changes. Each case has three questions: current state, previous state, and the exact date of the change. The episode body omits the date; the timestamp is supplied as episode metadata. Ground truth is written by the case generator, so it cannot leak from the evaluation label into graph construction. There are 60 questions per method. The synthetic scorer matches normalized entities and parses calendar dates.

The public dataset is the LongMemEval **Oracle** JSON, which supplies relevant sessions for each question. I selected 10 questions from each of knowledge-update, temporal-reasoning, and multi-session, using seed 5660 among cases whose combined session text did not exceed 30,000 characters. Sessions are ingested in timestamp order, one episode per session. The original paper used LongMemEval-S with much longer full histories and a different ingestion/evaluation setup. For this local subset, a DeepSeek judge marks whether a predicted answer is factually equivalent to the reference; questions, references, predictions, and contexts are retained for audit. No `has_answer` flags or gold answers enter the graph or answer prompts.

For every question, the same answer model and prompt receive one of three contexts:

- **Full history:** all dated conversation episodes.
- **Graphiti:** the top 10 retrieved facts and `valid_at`/`invalid_at` fields.
- **No temporal metadata:** the identical retrieved facts, with just those date fields removed.

The principal metric is accuracy (correct answers divided by evaluated answers). I also record answer input tokens, answer latency, graph search latency, graph build time, and Graphiti LLM calls. Graph ingestion cost is separate

from per-question answering cost. Each case has its own database, and every raw prediction is written to JSONL immediately after the case.

4. Results

4.1 Original reported result and comparability

In the paper's LongMemEval-S experiment, GPT-4o-mini with full context scored 55.4% versus 63.8% for Zep, with average answer context of roughly 115k versus 1.6k tokens. These values are reference points only. The Oracle subset, model, database, embedding model, retrieval context, and scoring judge in this project differ from that experiment.

4.2 Synthetic temporal behavior

Method	Current state (n=20)	Historical state (n=20)	Change date (n=20)	Mean answer input tokens
Full history	20/20	20/20	20/20	~144
Graphiti + validity dates	20/20	20/20	20/20	~234
Same facts without dates	20/20	20/20	0/20	~130

The 20/20 to 0/20 change-date drop is the measured effect of removing only date fields. It shows that the graph's temporal metadata reaches the answer stage. I repeated all 20 cases three times at temperature 0; all nine method-by-question-type accuracies were identical in every trial (observed standard deviation 0 percentage points). This repeated result is conditional on the same easy questions and does not establish performance on broader conversations. The full-history baseline is equally accurate and uses fewer answer tokens on these deliberately short cases; no efficiency improvement is claimed here. A two-event smoke test additionally showed an old employer fact invalidated on the new employer's start date.

4.3 LongMemEval Oracle subset

Method	Knowledge update (n=10)	Temporal reasoning (n=10)	Multi-session (n=10)	Overall	Mean answer input tokens	Mean retrieval + answer latency
Full history	9/10	6/10	9/10	24/30 (80.0%)	5,605	0.969 s
Graphiti + validity dates	3/10	2/10	2/10	7/30 (23.3%)	258	0.886 s
Same facts without dates	3/10	2/10	2/10	7/30 (23.3%)	187	0.878 s

Three questions are abstention questions whose reference answer says the information is insufficient. All three methods answered those correctly. Among the 27 answerable questions, the full-history baseline scored 21/27 and both graph conditions scored 4/27. Thus the aggregate graph score must not be read as successful retrieval on seven factual questions.

The graph condition cut answer prompt size by about 95.4%, but lost 56.7 accuracy points against full history. Its local query time was only about 0.08 s shorter on average; this figure excludes graph construction. Graph building took roughly 13.2 s and 10.7 DeepSeek calls per case on average, whereas the full-history baseline needs no graph build. The no-date ablation made no accuracy difference on this subset. This likely reflects a more basic bottleneck: answer-bearing facts were often absent from the retrieved edges, so temporal metadata could not help. For example, one knowledge-update question asked whether the user led four engineers initially and five now; the retrieved context only said that the user held a new Senior Software Engineer role, yielding `UNKNOWN`. Another question asked the price per mug; the full-history answer was $12 while the graph answer was `UNKNOWN`.

All 90 predictions completed without execution errors. The scores use one run of the 30-question subset. The public judge is itself an LLM and can make mistakes; raw gold answers, predictions, and contexts remain in `results/20261105-101232-longmemeval/predictions.jsonl` for manual review. The fixed seed, selection rule, and IDs are in that run's manifest.

5. Debug diary

- GitHub's ordinary clone endpoint timed out; GitHub's source archive API supplied a pinned snapshot.
- Docker and WSL were unavailable. Kuzu provided an embedded database, but upstream marks that backend deprecated.
- In Graphiti 0.31.2, `KuzuDriver.build_indices_and_constraints()` is a no-op even though full-text search expects those indexes. Calling the Kuzu maintenance operation directly created them.
- The current episode path also accesses a driver `_database` attribute that Kuzu lacks when a `group_id` is passed. A separate database per case avoids `group_id` and prevents cross-case contamination.
- The current source imports `httpx` without declaring it as a direct dependency; I installed it explicitly and pinned it in `requirements.txt`.
- The default Hugging Face endpoint was unreachable, so the embedding model was downloaded once through a mirror and then run from the local cache.

6. Limits and conclusion

This study demonstrates the intended temporal fact lifecycle and an isolated metadata ablation, but does not validate the paper's headline LongMemEval-S numbers. In the public subset, session-level ingestion compresses multiple messages into one episode, which can discard small answer-bearing details. The smaller embedding model, edge-only retrieval, Oracle selection, and different LLM/automatic judge also change the task. The synthetic cases are deliberately easy: the dates are present in metadata and each case has only two events. A larger follow-up should ingest individual messages, use the paper's exact split and prompts, include entity/community context, and repeat trials with manual review of judge disagreements. The raw outputs make those changes auditable.

References

- Rasmussen et al., Zep: A Temporal Knowledge Graph Architecture for Agent Memory, 2025.
- Authors' Graphiti repository, pinned to the snapshot above.
- LongMemEval cleaned dataset.